Routes to Safety

An evacuation-services map from open geospatial data — datasets, tiles, routing, rendering

DuckDB · Overture Maps · Planetiler · PMTiles · MapLibre GL JS · OSRM

Vishal Kumar · Shaishav Maisuria

FOSS4G NA 2026, Sacramento — Monday 2 November, 09:00–12:00, Room Beavis

github.com/vishal4c/routes-to-safety-workshop

What we work with — open datasets and open tools

OPEN DATASETS

OpenStreetMap
ODbL

the network we route on

Overture Maps
ODbL / CDLA-P 2.0

roads, places, towns, water, buildings

CAL FIRE FRAP
CC-BY

hazard zones, fire perimeters

ESA WorldCover
CC-BY 4.0

land cover at 10 m

OPEN TOOLS — one per phase

DuckDB ✦ spatial
Phase 1

SQL over files, local or S3

Planetiler → PMTiles
Phase 2a

GeoJSON → vector tiles → one static file

OSRM
Phase 2b, 2c, 3c

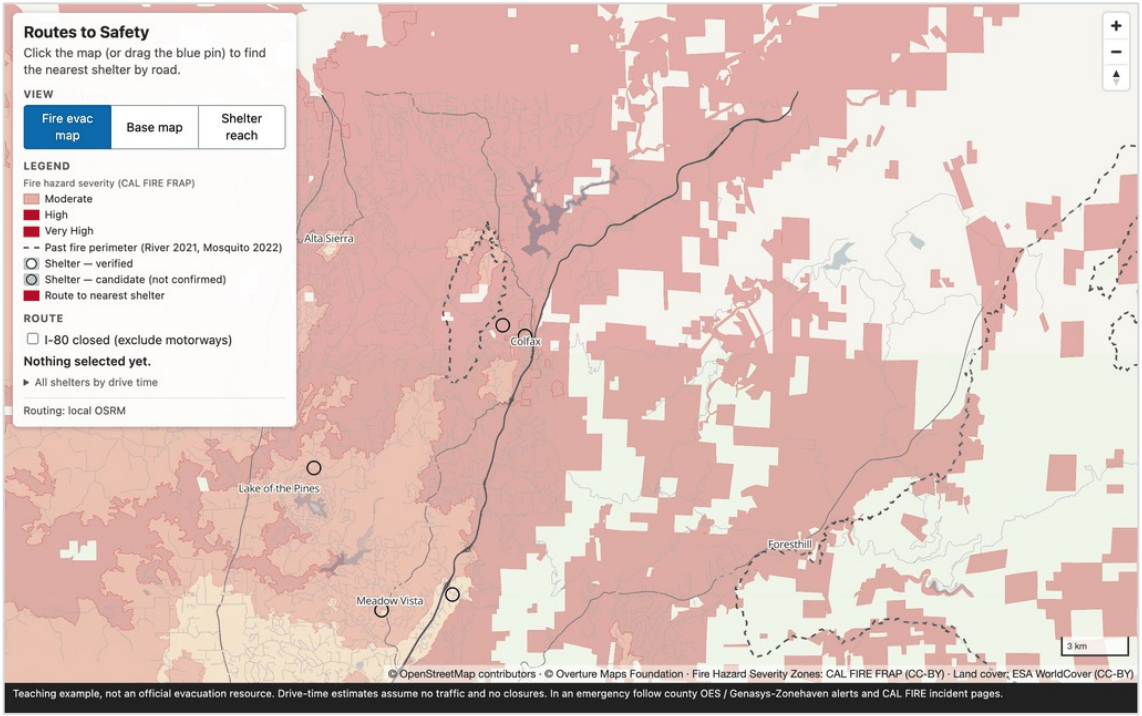
routing graph: nearest, table, route

MapLibre GL JS
Phase 3

draws tiles from a JSON style

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

The app in motion



1 Pin at Dutch Flat → Sierra Vista CC, 18 min

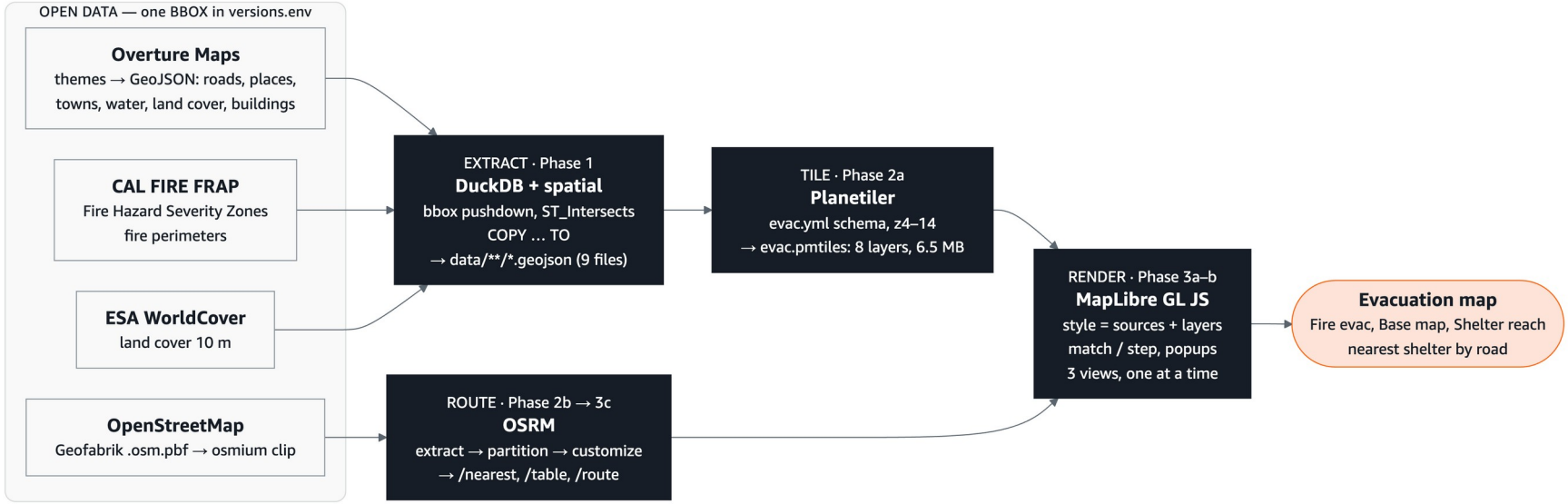
2 "I-80 closed" → same shelter, 31 min

3 Foresthill: nearest on the map ≠ nearest by road

4 Shelter reach: 10 / 20 / 30-minute bands

5 Base map: routing works in every view

The pipeline: one extract ⟶ a basemap and live routes



Phase 4 change one bbox line in versions.env ⟶ run the nine scripted steps ⟶ the same map for a town of our choice

Pinned	DuckDB 2.0.0	Planetiler 0.10.2	pmtiles 1.31.2	MapLibre GL JS 6.14	OSRM v26.10.0	verified by CHECKSUMS.sha256

Agenda — five phases, one map

0 **Setup**
pick a way to run, one checkpoint line

DuckDB

1 **Data prep**
explore, pick candidate shelters, export GeoJSON

DuckDB · Overture Maps

Break

2 **Map artifacts**
build the tiles; graph and reach bands prebuilt

Planetiler · PMTiles · OSRM

3 **Application**
style, three views, routing, close I-80

MapLibre GL JS · OSRM

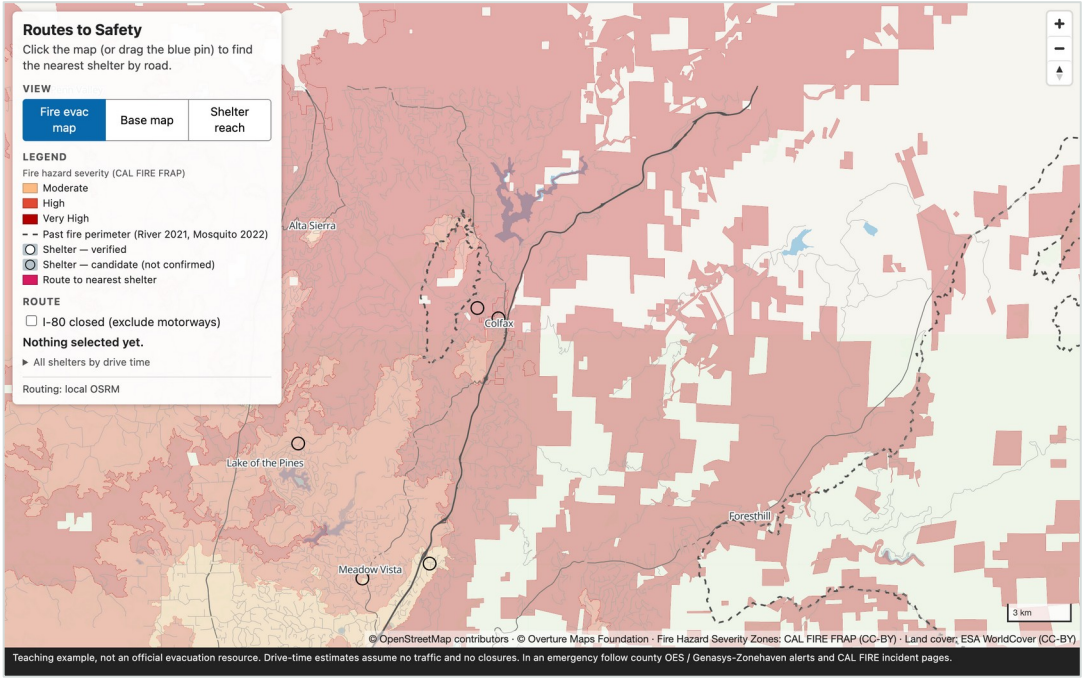
4 **Adapt**
one bbox, the same nine steps, our own town

versions.env · scripts/

Sticky note on the lid: **green** = fine, **red** = stuck

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Study area: the Placer County I-80 corridor



Shelters: 1 verified (Sierra College) + 9 candidates from Overture places. Hazard: CAL FIRE FHSZ 2024.

Rocklin → Auburn → Colfax → Dutch Flat / Foresthill

- bbox `−121.40, 38.72, −120.52, 39.20` (W, S, E, N)
- 36,294 road segments, 10 shelters

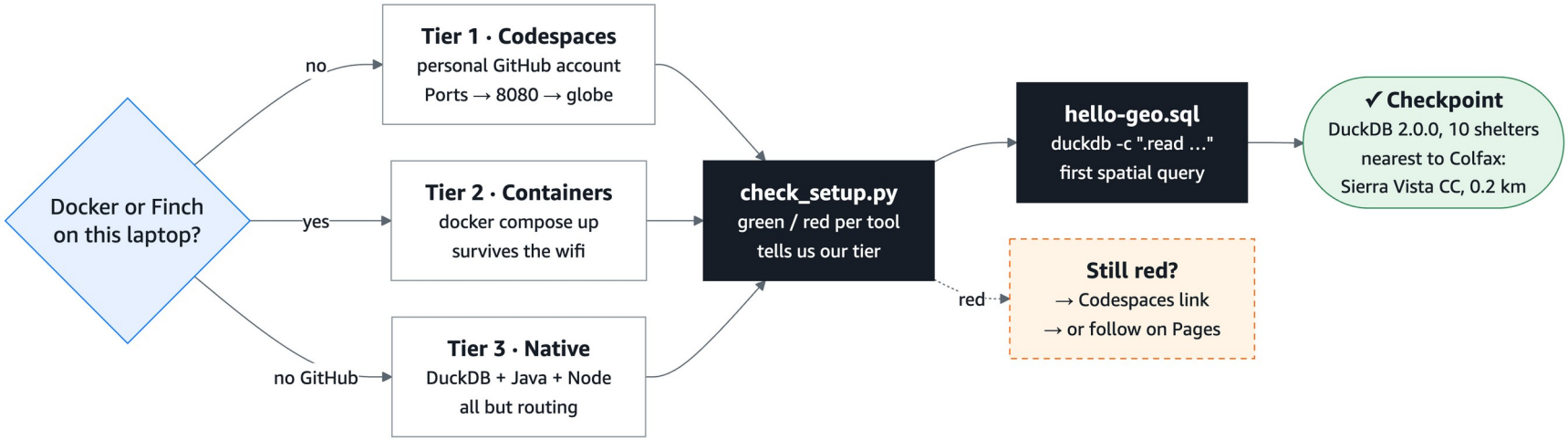
Why here

- River Fire 2021: Colfax evacuated
- Mosquito Fire 2022: the shelter moved mid-incident
- steep canyons: nearest on the map ≠ nearest by road

Not in this map

- shelter status and capacity
- live road closures
- drive time ≠ evacuation time

Phase 0 in one picture — pick a way to run, two lines, hands up



Phase 0 Docker or Finch on the laptop? → Codespaces, containers or native → two commands → hands up

✓ **Checkpoint** OK — DuckDB 2.0.0 · 10 shelters (1 verified, 9 candidate) · nearest to Colfax (crow-flies): Sierra Vista Community Center, 0.2 km

Three ways to run — pick one, keep a neighbour

Tier	Who	Routing	Notes
1 Codespaces	no Docker on the laptop	yes	**personal** GitHub account Ports tab → 8080 → globe
2 Local containers	**recommended** with Docker / Finch	yes	survives wifi loss corporate laptops: `finch compose up`
3 Native	DuckDB + Java + Node	no	Windows: Git Bash map and shelters only
4 Browser only	laptop failed	no	Pages site, follow along

PHASE 0 · setup checkpoint

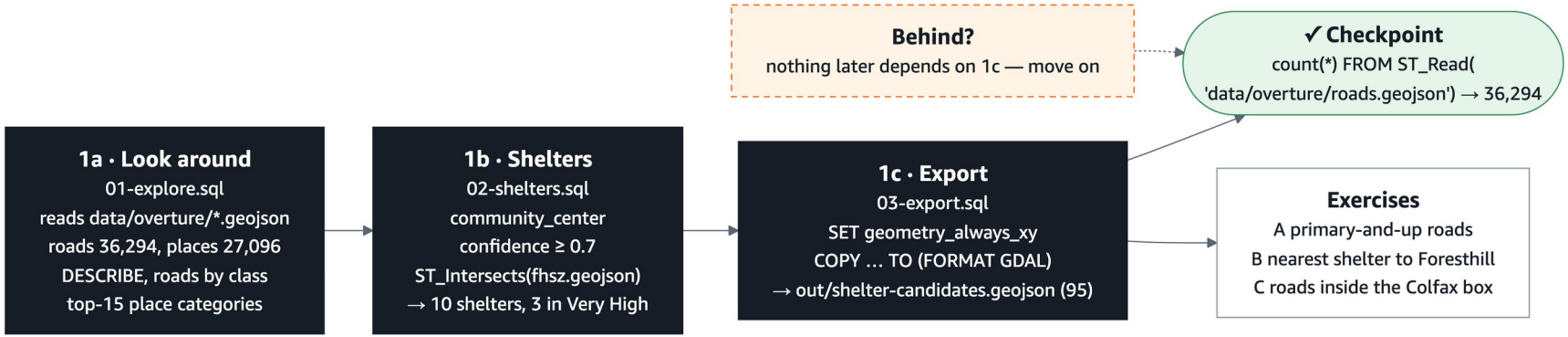
two lines from the repo root; hands up when they match

Still red? Codespaces. **Blocked?** Pages site.

```
python3 setup/check_setup.py
duckdb -c ".read setup/hello-geo.sql"
```

```
OK — DuckDB 2.0.0 · 10 shelters (1 verified, 9 candidate)
  · nearest to Colfax (crow-flies): Sierra Vista Community Center, 0.2 km
```

Phase 1 in one picture — three SQL files, one folder of GeoJSON



Phase 1 1a explore → 1b candidate shelters → 1c export

✓ **Checkpoint** SELECT count(*) FROM ST_Read('data/overture/roads.geojson') → **36,294**

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Overture Maps: five themes, one box



transportation

drivable road segments

36,294

roads



places

points of interest, with category and confidence

27,096

places



base

water and land cover

1,920 ✦ 2,564

water, land cover



buildings

footprints, height, floors

12,606

buildings (Auburn box)



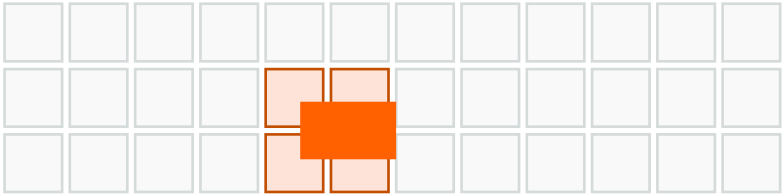
divisions

localities ⟶ town labels

121

towns

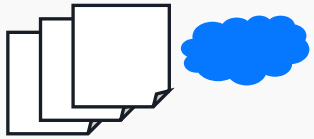
How a county comes out of the planet in 24 s



bbox per row ⟶ only the touched row groups are read

Overture	Linux Foundation	monthly GeoParquet	2026-09-23.1
Licence	ODbL (OSM)	CDLA-P 2.0 (places)	CC-BY (land cover)
Credit	© OpenStreetMap contributors	© Overture Maps Foundation	

DuckDB: one engine for all the data work



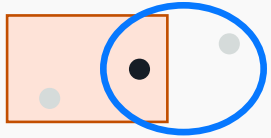
Reads files where they are

GeoJSON, Parquet, CSV; local or S3; no import



Prunes the planet to a box

bbox per row; only touched row groups are read



Asks spatial questions in SQL

a yes/no question about two shapes, inside WHERE



Writes the files the map needs

one COPY ⟶ GeoJSON for Planetiler and MapLibre

```
LOAD spatial;
SELECT s.name FROM shelters s, fhsz h
WHERE ST_Intersects(s.geom, h.geom) AND h.FHSZ_Description = 'Very High';
```

In the room one binary · no server · no account

Pinned DuckDB 2.0.0 · spatial · httpfs

Phase 1b: which shelters sit in a Very High zone?

Columns cheat sheet — raw Overture vs. our GeoJSON

We want	Raw Overture (S3)	Our data/overture/*.geojson
a place's name	names.primary	name
a place's category	taxonomy.primary	category
how sure Overture is	confidence (use >= 0.7)	same
road type	class, subclass	same
the shape	geometry	same

- fetch-overture.sh flattened the left column into the right
- no fairground category: Gold Country Fairgrounds hides elsewhere
- **Noise**: a Chevrolet dealer tagged community_center at 0.92

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

PHASE 1 · hands-on — data prep with DuckDB

1a · Look around

`duckdb -c ".read 00-data/01-explore.sql"`

three queries; the expected output is in the file

1b · Candidate shelters

`duckdb -c ".read 00-data/02-shelters.sql"`

community_center, confidence ≥ 0.7; spot the car dealer; ST_Intersects with FHSZ

1c · Export

`duckdb -c ".read 00-data/03-export.sql"`

⟶ out/shelter-candidates.geojson, 95 places

Behind? nothing later depends on 1c

```
LOAD spatial; SET geometry_always_xy = true;
COPY (SELECT id, name, category, confidence, geom
      FROM ST_Read('data/overture/places.geojson')
      WHERE category IN ('school','community_center', … ))
TO 'out/shelter-candidates.geojson' (FORMAT GDAL, DRIVER 'GeoJSON');
```

Exercises

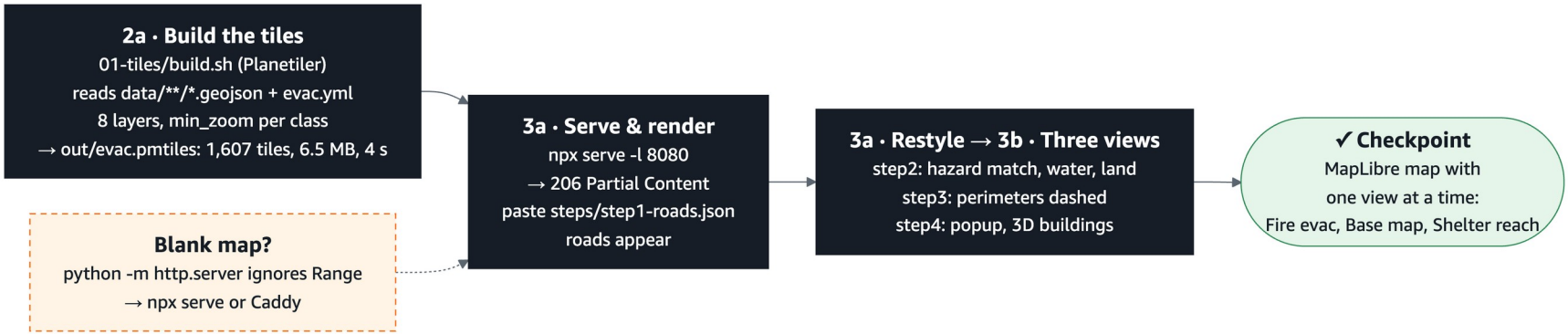
A motorway + trunk + primary ⟶ 1,909

B crow-flies nearest to Foresthill ⟶ Georgetown School, 12.4 km **(keep it for 3c)**

C finish early: roads inside the Colfax box

✓ **Checkpoint** `SELECT count(*) FROM ST_Read('data/overture/roads.geojson')` ⟶ **36,294**

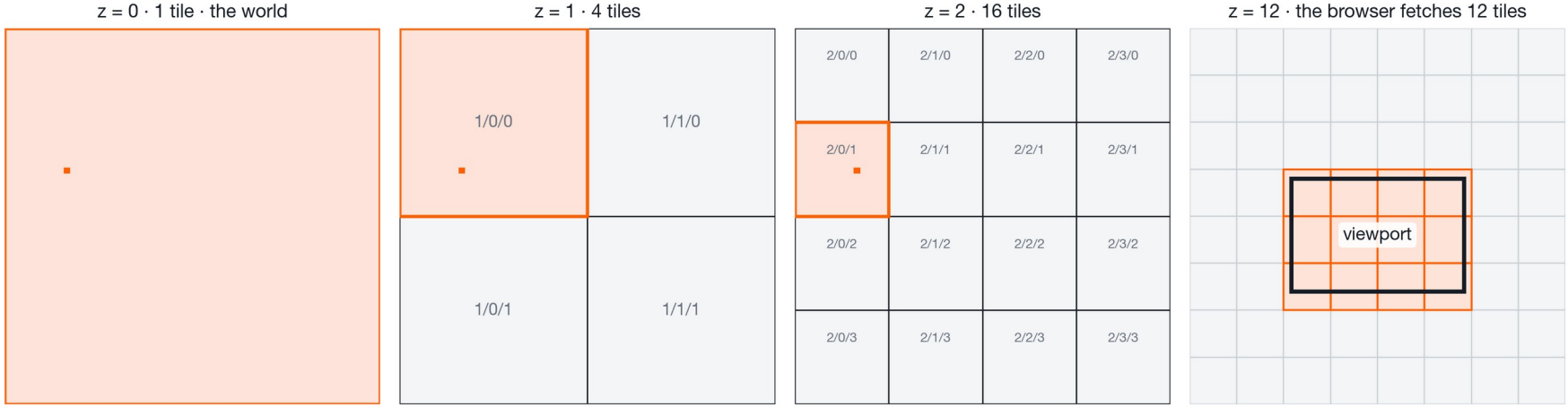
Phase 2 ⟶ 3 in one picture — GeoJSON ⟶ one tile file ⟶ a styled map



Phase 2a ⟶ 3b 2a build the tiles ⟶ 3a serve, render, restyle ⟶ 3b three views

✓ **Checkpoint** curl -sI -H "Range: bytes=0-15" http://localhost:8080/out/evac.pmtiles ⟶ **206 Partial Content**

Why tiles: the map is cut into a z/x/y quadtree



- **Each zoom splits every tile in four**: addressed z/x/y
- **The browser fetches only the tiles under the viewport**
- **Our file: 1,607 tiles, z4–14, 6.5 MB**, no database

z	tiles on Earth	over our box
4	256	1
8	65,536	4
10	1,048,576	12
14	268 M	1,218

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Vector tiles, PMTiles and the Range-request trap

- **Vector tiles**: geometry and attributes, drawn by the browser
- **PMTiles**: every tile in one file, fetched by byte range
- **No tile server**: GitHub Pages, S3, a USB stick
- **The trap:** `python -m http.server` ignores Range ⟶ blank map
- Use Caddy or `npx serve` ⟶ **206 Partial Content**

```
# the only test needed
curl -sI -H "Range: bytes=0-15" http://localhost:8080/out/evac.pmtiles | head –1
HTTP/1.1 206 Partial Content
```

```
# 01-tiles/evac.yml  (excerpt: 8 layers, all local files)
sources:
  roads: { type: geojson, local_path: data/overture/roads.geojson }
layers:
  - id: roads
    features:
      - { source: roads, geometry: line, min_zoom: 6,
          include_when: { class: [motorway, trunk, primary] } }
```

MapLibre style: sources, layers, and two expressions

```
"fill-color": ["match", ["get", "FHSZ_Description"],
  "Moderate",  "#fdbb84",
  "High",      "#e34a33",
  "Very High", "#b30 000",
  "#cccccc"]                    // fallback
```

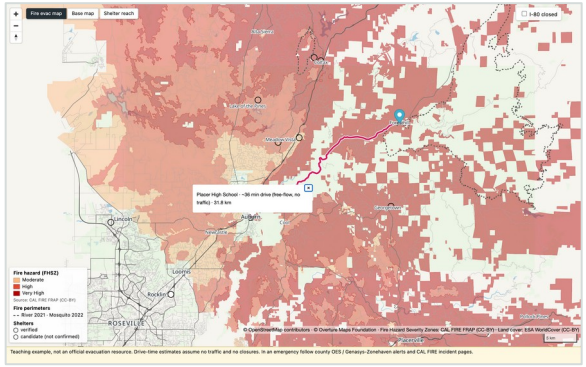
```
"line-width": ["step", ["zoom"], 0.6, 10, 1.2, 13, 1.6]
```

- `match`: one output per category
- `step`: one output per zoom range
- spelling must match: `"Very High"`, not `very_high`

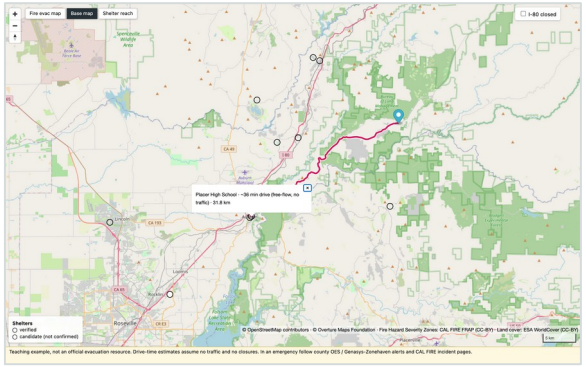


- **A style is JSON**: `sources` + `layers`, drawn bottom to top
- **Paste-in steps**: index.html orders and de-duplicates layers by id
- **Labels**: a symbol layer; glyphs vendored, works offline
- **Three views**: toggle layer visibility; the legend follows
- **Shelter reach**: GeoJSON source + `setFilter(shelter_id)`

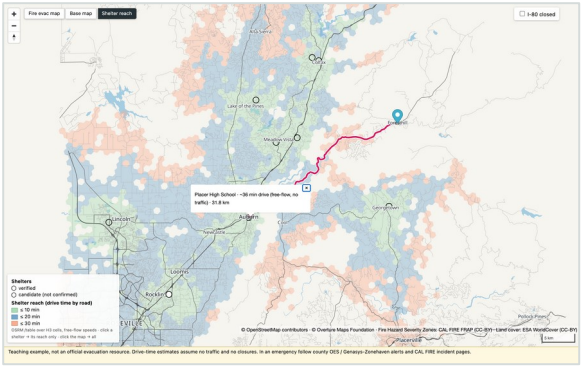
What Phase 3 produces: three views of the same tiles



Fire evac map: our tiles, hazard, perimeters, shelters, route



Base map: Esri street raster, shelters and route on top



Shelter reach: 10 / 20 / 30-minute bands per shelter

One view at a time

- the legend follows the view
- hazard edges are zone boundaries, not a guarantee

✓ Phase 3a checkpoint

- Very High zones solid red, water blue
- 10 shelter circles, 1 filled
- Mosquito perimeter dashed, city labels on
- attribution visible bottom-right

PHASE 2a ⟶ 3b · hands-on — build and style the map

Open the page

Codespaces Ports tab ⟶ 8080 ⟶ globe

Docker / Finch http://localhost:8080/02-web/

Native npx serve -l 8080 . ⟶ same URL

2a · Build the tiles

01-tiles/build.sh

pmtiles show out/evac.pmtiles

expect 1,607 tiles, z4–14, 8 layers, 6.5 MB

No file? cp data/prebuilt/evac.pmtiles out/

3a · Serve and render (step 1)

paste steps/step1-roads.json at // STEP 1

reload ⟶ grey roads, attribution bottom-right

3a · Restyle (steps 2–4)

step2: hazard, water, land cover, labels, shelters

step3 (if time): fire perimeters dashed

step4 (finish early): popups, 3D buildings

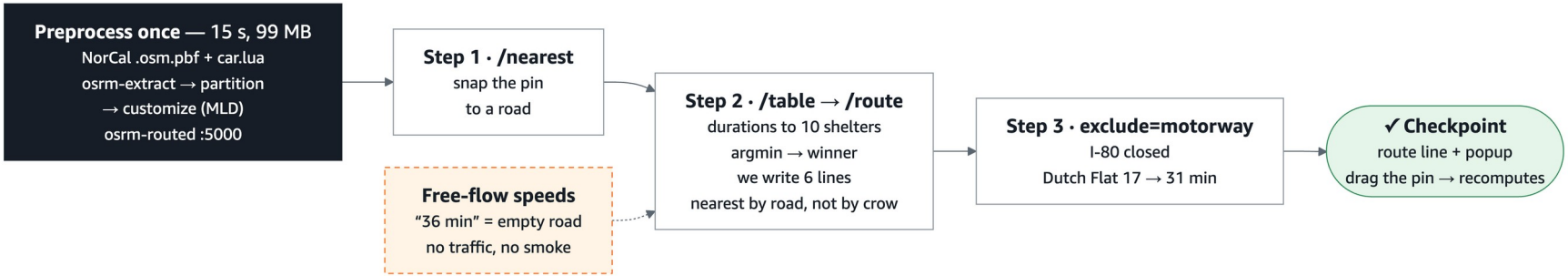
Stuck? steps/step3-styled.json lost? 02-web/solution.html

✓ **Checkpoint** Very High red, water blue, 10 shelters (1 filled), perimeter dashed

3b · Three views: Base map and Shelter reach buttons, one at a time

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 3c in one picture — preprocess once, then three HTTP calls

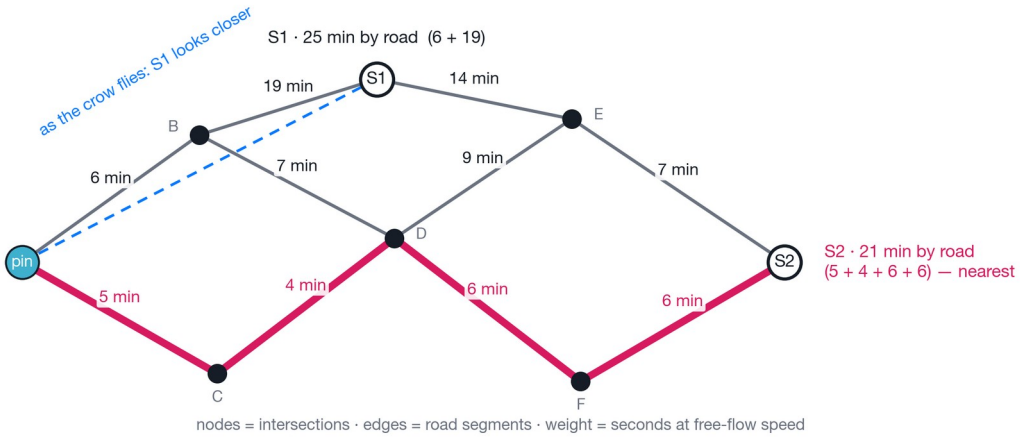


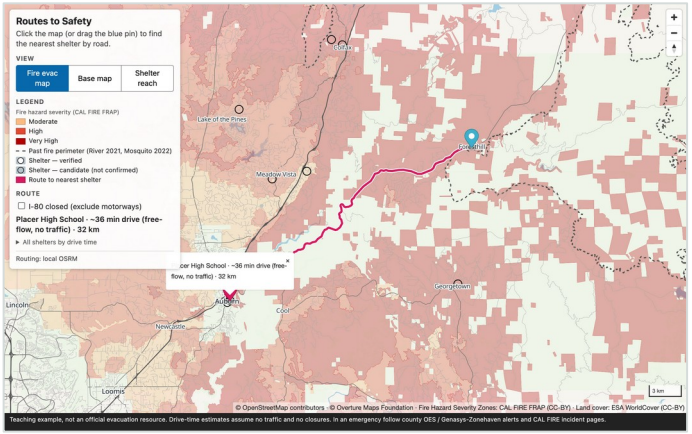
Phase 3c hello route (/nearest) → nearest by road (/table → argmin → /route) → I-80 closed (exclude=motorway)

Teaching numbers

Dutch Flat → Sierra Vista CC: 17.1 min, 31.3 with I-80 closed	Foresthill → Placer High: 35.8 min, 31.8 km

Why a routing graph: nearest on the map ≠ nearest by road





Foresthill: crow-flies says Georgetown (12 km); the road says Placer High (36 min)

The graph

intersections = nodes, road segments = edges, weight = seconds

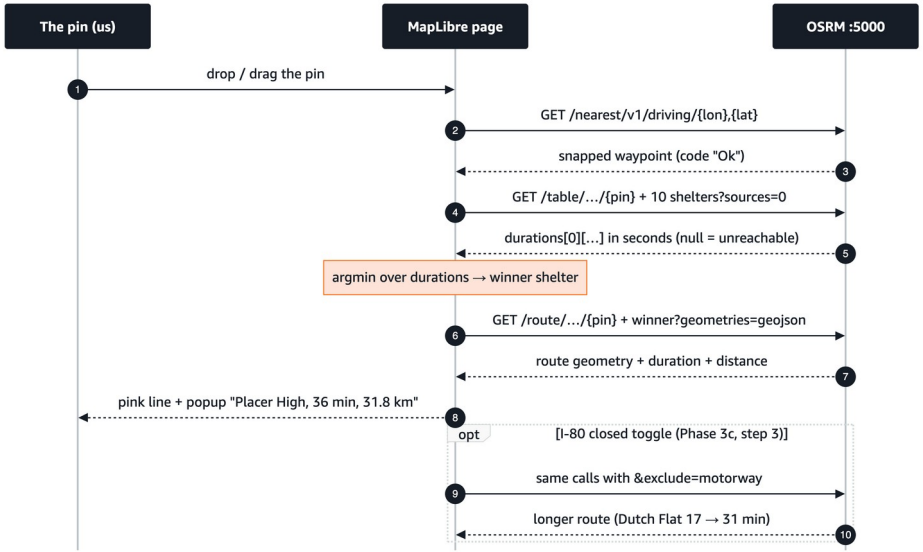
Dijkstra

cheapest frontier first, out from the pin, until a shelter

OSRM: preprocess once

extract → partition → customize: 15 s, 99 MB, then milliseconds

Routing is three HTTP calls



Call	Question it answers
/nearest	snap a coordinate to the nearest road point
/route	path, duration, distance between two points
/table	durations, many points to many points

```
GET $OSRM_URL/{service}/v1/driving/
    {lon},{lat};{lon},{lat}      # longitude first!
```

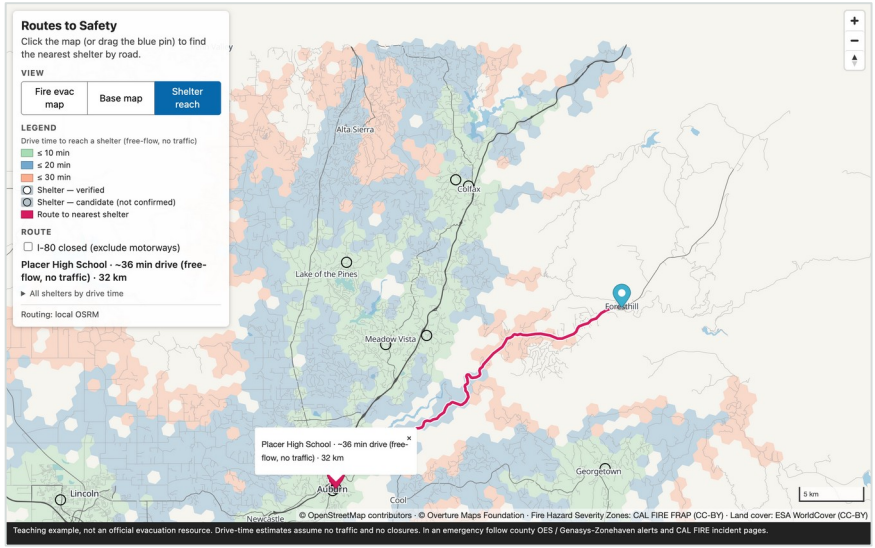
Free-flow speeds, not traffic: 36 min on an empty road

Tier	$OSRM_URL
1 Codespaces	<forwarded-8080-URL>/osrm (Ports → globe; never forward 5000)
2 Docker / Finch	http://localhost:8080/osrm (Caddy proxies to osrm:5000)
3 Native, 4 Browser	no routing: pair with a neighbour

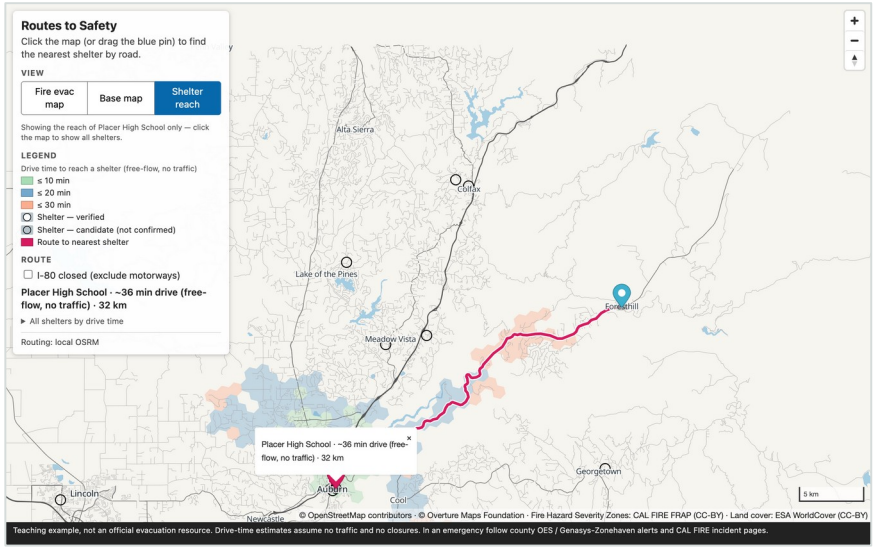
Snap, table, argmin, route: we write the argmin and the /route call

I-80 closed: the same calls with exclude=motorway

From routes to coverage: /table once, draw the reach



All shelters: 10 / 20 / 30-minute bands, dissolved H3 cells



Click a shelter: only its reach

make-coverage.py H3 grid over the box, 3,341 road cells → one /table call, cells × 10 shelters → DuckDB dissolves cells into bands

PHASE 3c · hands-on — routing

Step 1 · Hello route

`curl "$OSRM_URL/nearest/v1/driving/-121.08,38.90"`

expect "code":"Ok" and a waypoints array

then `steps/step1-route.js` → pink line, 35.8 min / 31.8 km

Step 2 · Nearest shelter by road

`steps/step2-scaffold.js`, six lines:

/table → argmin → /route → setData → popup

compare with Exercise B; drag either marker

Not routing? `steps/step3-solution.js`

Step 3 · I-80 is closed

`&exclude=motorway` toggle; from Dutch Flat: 17 → 31 min

A calculation, not detection: real closures are not a parameter

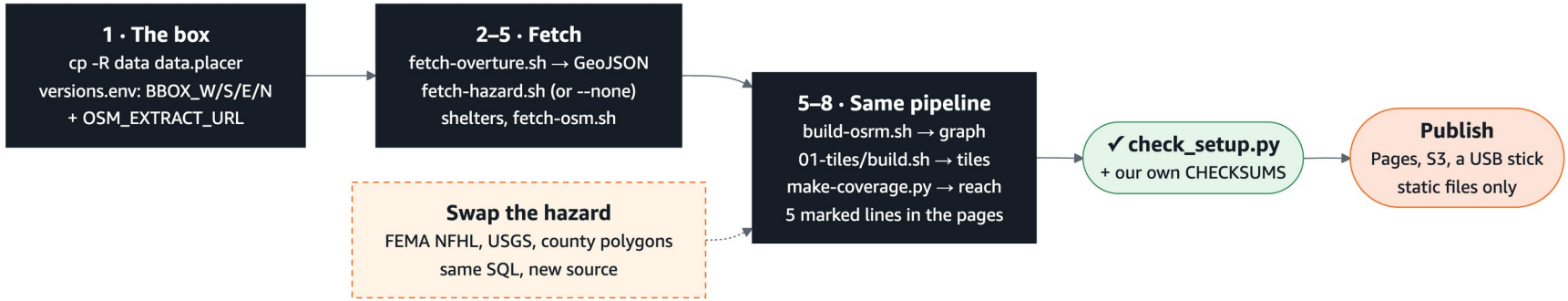
Tiers 3/4: pair with a neighbour

Stretch A: 15-min reach with grid.sql (400 points)

Stretch B: build the Colfax graph

✓ **Checkpoint** drag → line and popup update; the toggle changes the route

Phase 4 in one picture — one box, the same scripts, our own town



Phase 4 fetch-overture.sh → fetch-hazard.sh → shelters → fetch-osm.sh + build-osrm.sh → build.sh → make-coverage.py → 5 marked lines

Pinned DuckDB 2.0.0 | Planetiler 0.10.2 | pmtiles 1.31.2 | MapLibre GL JS 6.14 | OSRM v26.10.0

Reproducibility, then swap the community

Reproducible by construction

- `versions.env` pins every tool and image; CI fails on drift
- **Dated extracts**: Overture 2026-09-23.1, norcal 2026-10-22
- `CHECKSUMS.sha256` verifies the 143 MB bundle
- **One script per step**:

`fetch-overture.sh` `fetch-hazard.sh` `fetch-osm.sh`

`build-osrm.sh` `01-tiles/build.sh` `make-coverage.py`

PHASE 4 · Swap the community

1. cp -R data data.placer; edit `BBOX_*` and `OSM_EXTRACT_URL`

2. `fetch-overture.sh`, `fetch-hazard.sh` (`--none` outside California)

3. shelters → `fetch-osm.sh` + `build-osrm.sh` → `build.sh` → `make-coverage.py`

The OSM download is the big one: 100 MB – 1 GB

Swap the hazard

- FEMA flood zones: new polygons, same SQL and style
- **Live use adds**: status, closures, capacity

Laptop ⟶ planet: same shape, bigger inputs

Stage	Today (0.4 sq°)	At scale
Extract	DuckDB bbox pushdown, 24 s, 38 MB GeoJSON	same SQL, one region at a time
Tiles	Planetiler, 4 s, 6.5 MB PMTiles	planet builds are a documented Planetiler use case
Serve	Caddy or npx serve on :8080	PMTiles on object storage behind a CDN
Routing	OSRM MLD graph, 15 s, 99 MB	one graph per region; Valhalla / GraphHopper for isochrones
Freshness	pinned release 2026-09-23.1	monthly releases: bump the pin, rerun, diff

In the wild

- the same tools run router.project-osrm.org and the Protomaps basemaps
- **Not here**: live closures, shelter status, capacity, traffic

Recap — the pipeline we built



- **Open data, open tools**: no accounts, no servers, re-runs on a laptop
- **Two questions, one extract**: the safe places, and how to reach them
- **Free-flow only**: no status, traffic, closures or capacity
- **Keep**: the repo, the 143 MB bundle, the demo, one script per step

Licences, attribution, links

What	Licence
Code	MIT
Materials (slides, docs)	CC BY-SA 4.0
OpenStreetMap	ODbL
Overture Maps	ODbL / CDLA-Permissive 2.0 (by theme)
CAL FIRE FHSZ, perimeters	CC-BY
ESA WorldCover	CC-BY 4.0
Noto Sans glyphs	SIL OFL 1.1

Repo github.com/vishal4c/routes-to-safety-workshop

Demo vishal4c.github.io/routes-to-safety-workshop/demo/

Data bundle .../releases/tag/data-v2 (143 MB; scripts/get-bundle.sh)

Feedback form [URL / QR to add]

Learn more

- duckdb.org/docs/extensions/spatial
- docs.overturemaps.org
- github.com/onthegomap/planetiler
- docs.protomaps.com/pmtiles
- maplibre.org
- project-osrm.org/docs

Attribution to keep on every map

- © OpenStreetMap contributors
- © Overture Maps Foundation
- Base map: © Esri, HERE, Garmin, USGS
- Fire Hazard Severity Zones: CAL FIRE FRAP (CC-BY)
- Land cover: ESA WorldCover (CC-BY)

Thank you.

Vishal Kumar and Shaishav Maisuria — questions: stay, or find us in the hallway.

Teaching example, not an official evacuation resource. Drive-time estimates assume no traffic and no closures.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0